

Design Evaluation of Self-Learning Line Following Robot

Nadia Camacho, Jared Butcher, Nate Kanyok, Xiangxu Lin, Jong-Hoon Kim

Abstract—The development of a line-following robot is, for many, a building block into further applications of robotics. One of the main focuses is pathfinding without prior knowledge of the route or environment. In the case of a simple line-following robots, the task is limited, allowing for much room in experimenting with different approaches. In this paper, we present our dual-control line following robot - ATR LineBot. The first control mechanism uses two series of line sensors in combination with a custom PID controller for speed optimization. The additional sensors allows for sophisticated planning and correction to be done. Our second control mechanism utilizes a double layered neural network for remote learning of optimal speed. Utilizing the output of line detection sensors and motor encoder as feature vectors, the neural network can optimize control. ATR LineBot, the dual-control line following robot, uses both PID controlling and reinforcement learning for successful navigation and serves as a platform for improving the traditional building block of robotics. Thus, we present a smart line following robot comprising a robust design, which is compatible with any circumstance as well as efficient in achieving its ideal speed to perceive its goal.

Index Terms—Line following robot, Pathfinding, PID control, reinforcement learning.

I. INTRODUCTION

Line following robots are typically one of the first steps many take when they first approach robotics. That is why many starting robotic kits are designed towards fulfilling it. Learning the basics of path finding are built on top of it. At first, the robot would be configured to move if it stops a line and stop to go back in reverse if it no longer detects it. Or perhaps, instead of moving bidirectionally, it would just turn until it spots the line again. Quick analysis would prove that this method is rudimentary, and could be improved. However, the methods to improve it are diverse.

The most common is through utilizing a PID which is what we will discuss. In addition, we have a developed a simulator that allows for quick testing and refining of PID parameters.

II. RELATED WORK

Several other researchers have looked into line following robots. First there has been focus on how the line should be detected. One of these ways is through the use of photo reflectors. Then there is also the topic of how sensors should be distributed. Many sensors would lead to a higher precision of accuracy of the line, but at the cost of the overall efficiency of the robot due to several factors. Generally at least two sensors were desired with the intent of having them separated an equal distance apart from one another yet still centered [1]. The

line can also be detected through a typical camera. Using rgb values, one would focus on color detection [4].

Yet the other main focus is through the steering. There have been some that focused on PID. Others have turned towards neural networks using a minimal amount of neurons to fasten the learning process. They also discuss other strategies such as reinforcement learning as well as using vector quantization [2]. There is even research between a combination of the two, known as a neural network PID.

Ultimately, the purpose of learning all of these is eventually use them in the field. For starters many become involved in line following robots because many competitions use them. And the reason they are so prevalent in competitions is because this work can be applied for various applications ranging from military to transportation to tending those with special needs [3].

III. PROPOSED APPROACH AND DESIGN

The design of line following robots are generally similar. Each robot must have wheels, sensors, a base, etc. However, the specific components used vary dramatically, along with their results.

A. Hardware

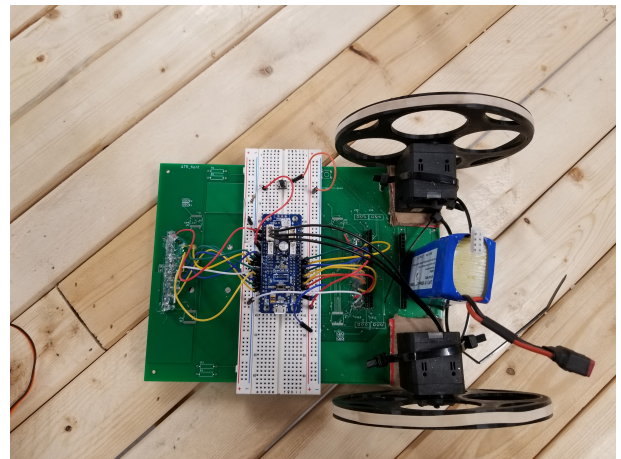


Fig. 1. line following robot

First we will start from the exterior before working our way towards the center. We chose the design of having two wheels in the back and one castor wheel in the front for several reasons. Having the castor wheel in front allowed a greater ease of changing directions with little resistance.

Also only having one dynamixel for each wheel simplified the specified output the PID would give. The dynamixel is what moves the robot using the power from the battery. The less the PID controller needs to focus on, the more we could optimize it with less hassle. We did not choose a four wheel design because doing so would also require getting more wheels, as well as more dynamixels which would only increase the weight, thus slowing down the robot. Furthermore, the two wheels we were using fulfilled the utility of getting the maximum distance while still be able to have the sensors be able to read the line. We were limited by the sensors only being able to be a few centimeters from the surface. The base of the robot was tilted with the sensors as close to the ground to maximize accuracy in reading the line. Then a 3d model of a wheel was modified to fit our specifications, to maintain the higher height the back needed.

The sensors are lined up right at the front by the castor wheel. Initially, we were going to use a multiplexor but because we could not have one, we opted to instead use only eight sensors lined up front. Had we had a multiplexor, we would have been able to use a much larger array of sensors and still be able to output a single signal to determine the next instruction to take.

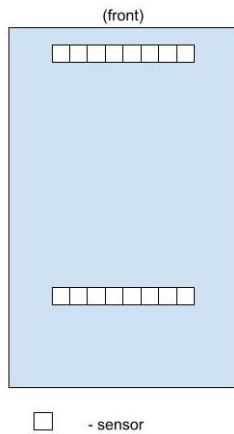


Fig. 2. software diagram

All of the movement of the line following robot is dictated by the opencm running on top of a breadboard. Initially we were going to use pcb and solder the rest but due to some difficulties such as the opencv not fitting in properly, we were faced with the alternative of using the breadboard and use jump wires. Having to adjust a project given a timeline and unforeseen difficulties is part of the computer science development experience.

B. Software

The software used to control the robot is implemented in C++ and is written to be compilable for both the OpenCM micro controller as well as traditional computers. The difference between the two compilation options is that compiling for OpenCM will have the motor control and IR sensor

interfaces manipulating the physical components. Maps are given to the simulator in to form of bitmap images, allowing easy visualization and creation. The simulator will judge the accuracy of the simulated line following robot by keeping a ratio of time on the line vs time off the line as well as time it has taken to complete the course.

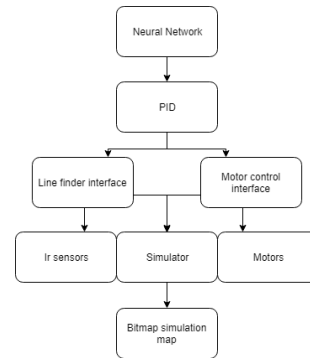


Fig. 3. software diagram

1) *PID*: A PID is a technique used to regulate an output of an analog system to a specified setpoint using inputs from said system. PID stands for Proportion, Integral, and Derivative. These three components will modulate the output according of its displacement from the setpoint, how long and far its been off setpoint, and its predicted future displacement respectively. The net result of which is a smooth and reactive response to environmental stimulus. A critical element of PID control is the tuning of the PID values. These determine the strength of the response of the three components and its effects on the output. In the case of a line following robot tuning is a matter of maximizing overall speed while at the same time still being able to make the required turns. If the strength is too high it will oscillate back and forth and potentially turn itself around, if too weak it will leave the line entirely on sharp turns. The most basic way to optimize a PID is to adjust its values while the robot is physically running and observe its behavior. This approach has some clear flaws in both time consumption and accuracy. PIDs also tend to be optimized for specific environments, for example, the optimal settings for a track filled with very sharp turns will have the robot moving more slowly and more strongly then one with gentle curves. So if optimizing using the manual approach mentioned above then it will require significant time testing for each environment. Also even if it is prepared for multiple environments it will have to told which environmental settings to use. Certainly something not practical in the field.

C. Development Process

The robot was developed in stages starting with the design of the electronics and basic PID software to make a working prototype.

1) *Electronic Plan*: Its detection was planted to be two rows of 32 IR sensors each, one in the front and one in the middle. The propulsion was two Dynamixel servo motors, in

the back with one on each side along with a coaster in the front middle. The control is an OpenCM microcontroller and four multiplexes to allow the use of all 64 IR sensors. A UART Bluetooth module and buttons were used for control. All of the circuitry was made into a printed PCB and powered by a battery.

2) *Robot Implementation:* In the end because of difficulty soldering the multiplexes to the board only 16 IR sensors were used. Eight on each row. These were wired directly to the microcontroller. It only implemented a single push button that was used to turn the robot on/off.

3) *Simulator Implementation:* The simulator was implemented with the ability to read bitmap images as tracks and simulate the robot in the environment. Even though it didn't take momentum into account, it was quite accurate at predicting the robot's performance. This was because the robot was driven by two low top speed and high torque servos. The final implementation of the simulator supported a limited ability to search for an optimal P value of the PID. It does this repeatedly testing the simulated robot on the track, rating its performance, then adjust the P value in the direction that the trend shows the rating improving. To avoid local maximums this adjustment has some momentum to purposely overshoot. The magnitude of this momentum also adjusts when it overshoots and returns to allow it to stabilize on an optimal.

IV. DISCUSSION

This paper presents an application of introductory computer science methods to a line following robot. Line following robots are often used as a learning platform in academia. The availability of kits allow for hands-on experiences in classrooms. However, there exists additional educational components of these basic robots. We have presented two improvements (simulator and PID control) to the traditional line-following robot that expand its applicability in an educational setting.

A. Achievements

We have successfully demonstrated PID control and a simulator of a custom-designed line following robot. This robot successfully performed in a university-wide competition, placing second place. We lost to a robot that was ultimately faster at going through the track. However our robot had a bigger focus on tracking the line and never went off track. Through testing of PID inputs via the simulator, we have improved our performance since the competition.

B. Going Forward

We have two immediate plans to improve this project going forward. The first is to re-construct the robot using the PCB board as intended, fully utilizing multiplexers and power management. The second is to implement a neural network so that the robot will be able to improve its PID performance and more successfully navigate a given track.

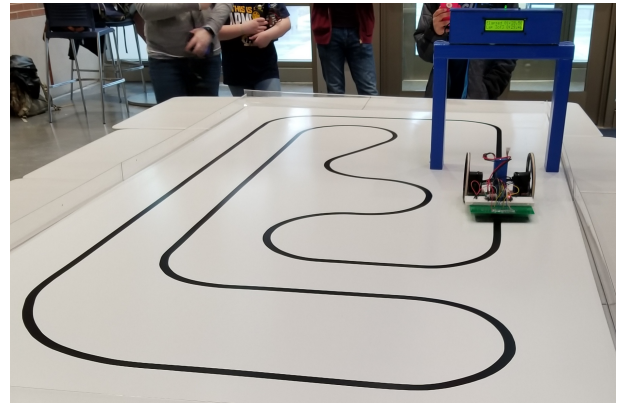


Fig. 4. robot at line following competition

V. CONCLUSION

A lot was learned with this robot from its manual construction to its implementation. Line following robots are a good entry point in understanding PID controllers, as well being a good step towards designing and building a robot without the use of a kit.

ACKNOWLEDGMENT

The authors would like to thank all the support they received from ATR, the Advanced Telerobotics Lab at Kent State University.

REFERENCES

- [1] M. Zafri Baharuddin and Izham Z. Abidin and S. Sulaiman Kaja Mohideen and Yap Keem Siah and Jeffrey Tan Too Chaun, *Analysis of Line Sensor Configuration for the Advanced Line Follower Robot*, 2001
- [2] Abhishek Roy and Matthew Mitra Noel, *Design of a high-speed line following robot that smoothly follows tight curves*, in *Computers & Electrical Engineering*, 2015.
- [3] Mehran Pakdaman and Mohammad Mehdi Sanaatiyan and Mahdi Rezaei, *A line follower robot from design to implementation: Technical issues and problems*, 2010.
- [4] M. V. Gomes and L. A. Bssora and O. Morandin and K. C. T. Vivaldini, *PID control applied on a line-follower AGV using a RGB camera*, in *2016 IEEE 19th International Conference on Intelligent Transportation Systems*, 2016.